# DDC Client JS SDK Wiki @Ulad Palinski

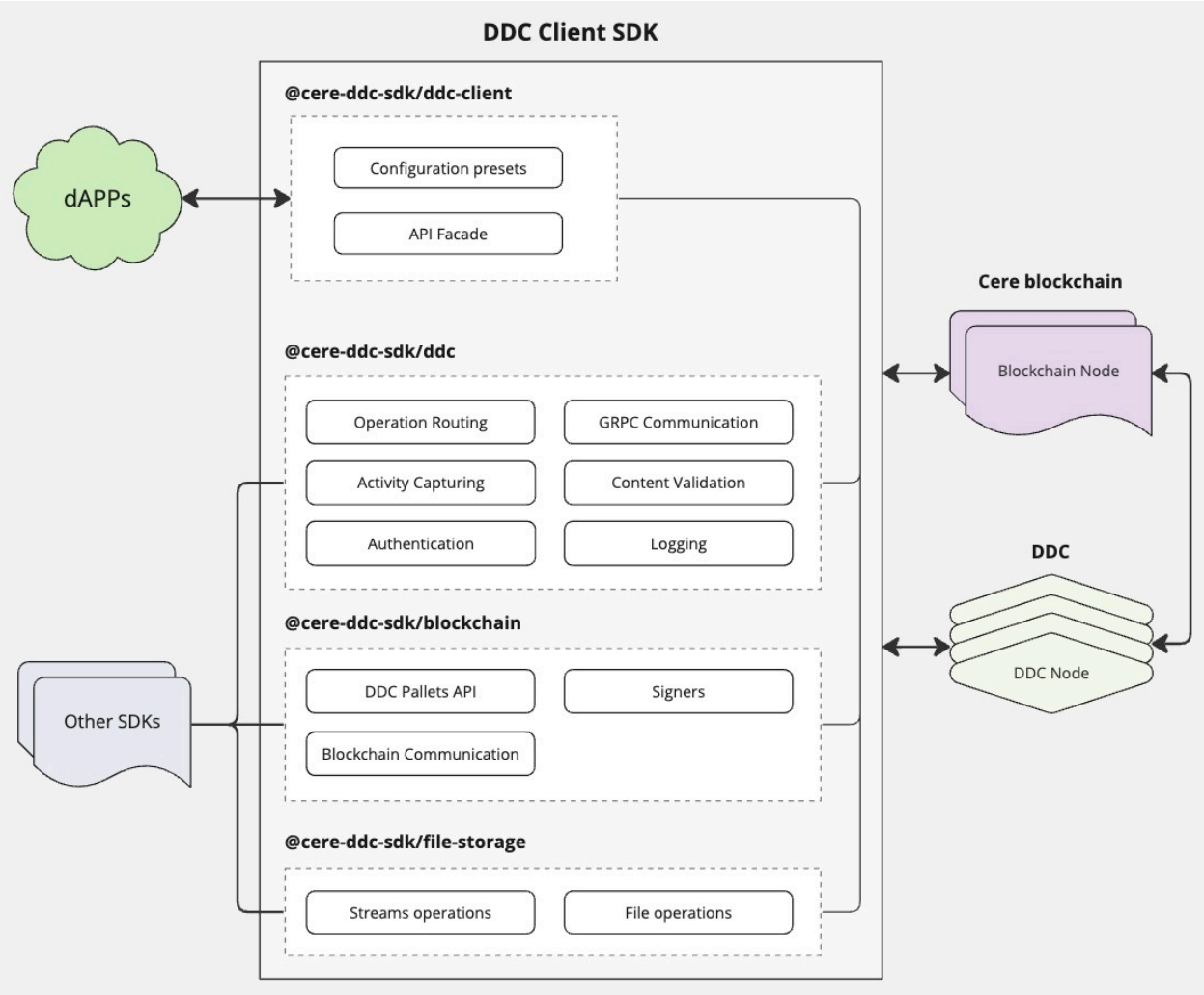
## 👋 Description / Purpose

The DDC SDK is a development kit used by developers to create applications that interact with the CERE infrastructure. It provides a set of modules and methods that allow seamless integration with the Cerebellum Network's decentralized data clusters (DDC).

### Definitions

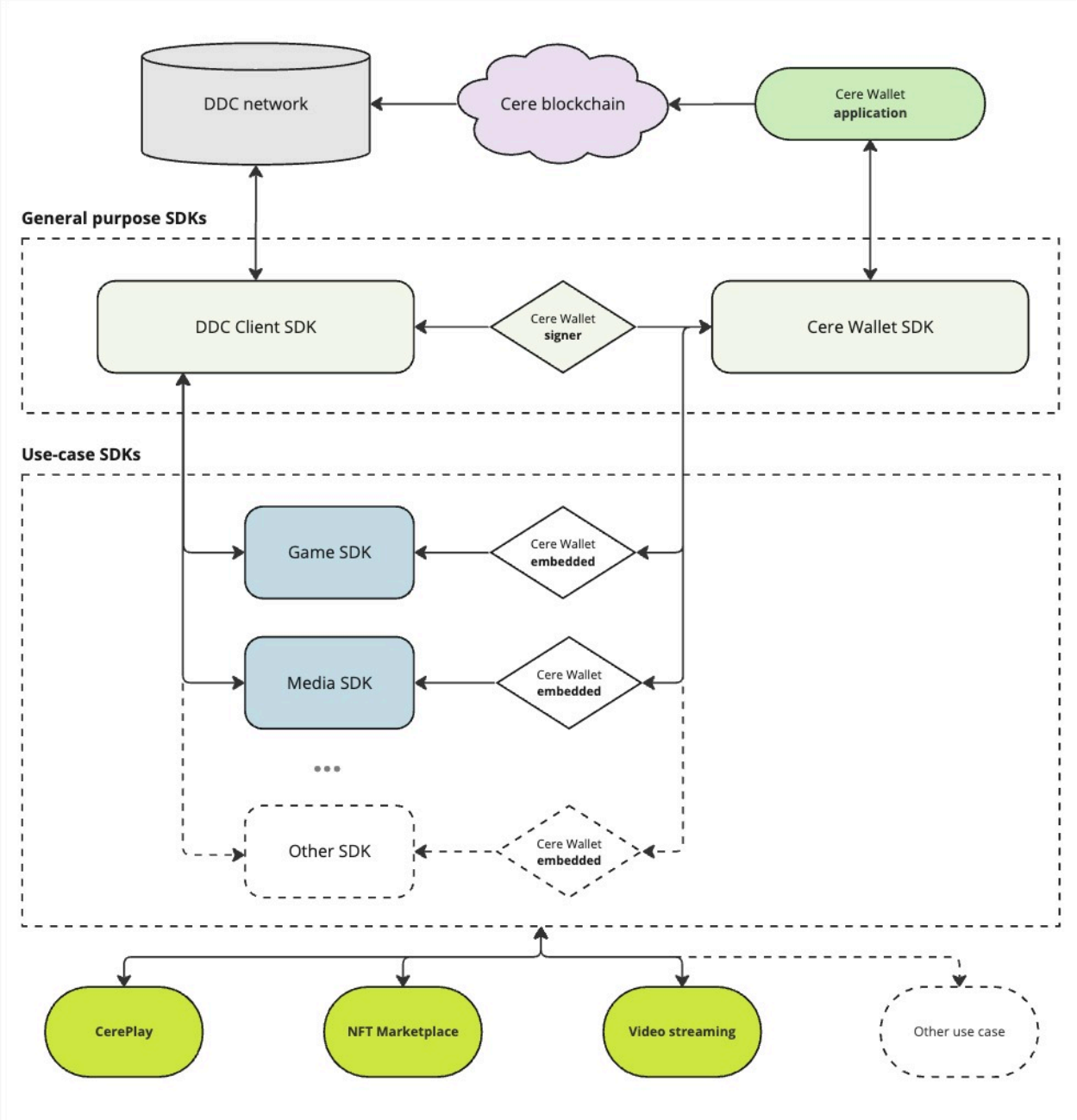
- **DDC** - Decentralized Data Cloud, a product of the Cerebellum Network for data storage.
- **Storage Node** - A fundamental component of the DDC that enables storing, reading, and searching data, like objects, files, and documents within the Cere Network.
- **CID** - Content Identifier, based on the content's cryptographic hash label used to point to material. It doesn't indicate where the content is stored, but it forms an address based on the content itself. CIDs are short, regardless of the size of their underlying content.
- **Piece** - An abstraction that signifies a unit of data stored in the DDC. It doesn't have a fixed size and can represent fully logically complete data or a part of it.
- **CNS** - Content Name System to associate CIDs with human-readable names.
- **DAG** - Directed Acyclic Graph stored on DDC is used to establish data relationships between different pieces of data.

## 🛠️ Technical Specifications




The SDK provides several modules. The `DDC Client` module acts as an entry point and offers a concise API for straightforward use cases, like creating a bucket or uploading/downloading a file. Additional modules present more advanced APIs that provide enough flexibility for other SDKs to be built on top of the DDC SDK.

# Use Cases




https://miro.com/app/board/uXjVN-LeM_c=/?moveToWidget=3458764577262442396&cot=14

The diagram above illustrates the role of the DDC SDK as a general-purpose system component. It is used in a variety of cases, such as CerePlay, the NFT Marketplace, and it operates behind the scenes in Media and Game SDKs.

## Specifications

Client side routing

Cere Wallet integration

## Authentication

The SDK authenticates by signing requests with a user key (derived from seed / wallet). Blockchain transactions are signed and sent to the chain endpoint; storage/compute requests to DDC nodes are signed by the same identity and verified by the node. Authorization is enforced based on bucket permissions (public/private) and on-chain customer/bucket state.

# 📦 Repositories

- https://github.com/Cerebellum-Network/cere-ddc-sdk-js/

  This is a mono-repository project that includes all the DDC SDK modules.

# 🚀 Releases

- https://github.com/Cerebellum-Network/cere-ddc-sdk-js/releases

# 👷Deployments

Since the SDK is an NPM package - it is not deployed anywhere. It is published to NPM instead:

- @cere-ddc-sdk/ddc-client
- @cere-ddc-sdk/blockchain
- @cere-ddc-sdk/ddc
- @cere-ddc-sdk/file-storage
- @cere-ddc-sdk/cli

# 📚Documentation

For detailed documentation and API references got to GitHub READMEs

# 💁 How to guides

The list of guides demonstrating how to use different SDK modules:

▼ DDC Client

A quick guide of how to upload a file to DDC TESTNET using the `DdcClient` API.

1. Create a `DdcClient` instance

```
import * as fs from 'fs';
import { DdcClient, File, TESTNET } from '@cere-ddc-sd
k/ddc-client';

const seed = 'hybrid label reunion only dawn maze asset
draft cousin height flock nation';
const ddcClient = await DdcClient.create(seed, TESTNE
T);
```

> The account used to create the instance should have positive balance and DDC deposit

2. Create a new public bucket

```
const bucketId = await client.createBucket(clusterId, {
isPublic: true });
```

3. Upload a file to DDC

```
const filePath = './my-picture.jpg';
const fileStats = fs.statSync(filePath);
const fileStream = fs.createReadStream(filePath);
const file = new File(fileStream, { size: fileStats.siz
e });

const { cid: fileCid } = await ddcClient.store(bucketI
d, file);
```

```
console.log('The uploaded file CID', fileCid)
```

4. That is it. You can open the file from your browser:

```
const fileUrl = `https://storage.testnet.cere.network/
${bucketId}/${fileUri.cid}`;

console.log('The file URL', fileUrl);
```

or download it using the SDK

```
const uri = new FileUri(bucketId, fileCid);

const fileResponse = await ddcClient.read(uri);
const content = await fileResponse.arrayBuffer();

console.log(content);
```

▼ File Storage

A quick guide of how to upload a file to DDC TESTNET using the `FileStorage` API.

1. Create a `FileStorage` instance

```
import * as fs from 'fs';
import { FileStorage, File, TESTNET } from '@cere-ddc-s
dk/file-storage';

const bucketId = 1n;
const seed = 'hybrid label reunion only dawn maze asset
draft cousin height flock nation';
const fileStorage = await FileStorage.create(seed, TEST
NET);
```

> The account used to create the instance should have a positive balance, DDC deposit, and the bucket should be created in advance

2. Upload a file to DDC

```
const filePath = './my-picture.jpg';
const fileStats = fs.statSync(filePath);
const fileStream = fs.createReadStream(filePath);
const file = new File(fileStream, { size: fileStats.siz
e });

const fileCID = await fileStorage.store(bucketId, fil
e);

console.log('The uploaded file CID', fileCID)
```

3. That is it. You can open the file from your browser

```
const fileUrl = `https://storage.testnet.cere.network/
${bucketId}/${fileCID}`;

console.log('The file URL', fileUrl);
```

▼ Blockchain

An example how to create a bucket using the `blockchain` package

1. Create a `Blockchain` instance and connect it to TESTNET

```
import { Blockchain, UriSigner } from '@cere-ddc-sdk/bl
ockchain';

const account = new UriSigner('//Alice');
const blockchain = await Blockchain.connect({
  wsEndpoint: 'wss://rpc.testnet.cere.network/ws',
});
```

2. Make a deposit

```
const deposit = 100n * 10n ** blockchain.chainDecimals;
// 100 CERE
const tx = blockchain.ddcCustomers.deposit(deposit);

await blockchain.send(tx, { account })
```

> The account used to make deposit and create the bucket should have positive CERE tokens balance.

3. Create a public bucket

```
const clusterId = '0x...';
const tx = blockchain.ddcCustomers.createBucket(cluster
Id, { isPublic: true });

const { events } = await blockchain.send(tx, { account
})
const [bucketId] = blockchain.ddcCustomers.extractCreat
edBucketIds(events);

console.log('Bucket ID', bucketId);
```

▼ DDC (low level API)

An example of how to upload a small piece of data using the `StorageNode` API.

```
import { StorageNode, StorageNodeMode, UriSigner, Piece }
from '@cere-ddc-sdk/ddc';

const signer = new UriSigner('hybrid label reunion only da
wn maze asset draft cousin height flock nation');
const storageNode = new StorageNode(signer, {
  mode: StorageNodeMode.Storage,
  logLevel: 'debug',
```

```
  grpcUrl: 'grpc://localhost:9091',
  httpUrl: 'http://localhost:8091',
});

const bucketId = 1n;
const content = new TextEncoder().encode('Hello DDC');
const piece = new Piece(content);

const cid = await storageNode.storePiece(bucketId, piece);

console.log('Stored piece CID', cid);
```

▼ CLI

An example of how to upload and download a file using DDC CLI

- Upload a file

```
npx @cere-ddc-sdk/cli upload ./picture.jpg
    --signer "hybrid label reunion only dawn maze asset
draft cousin height flock nation"
    --clusterId "0x825c4b2352850de9986d9d28568db6f0c023
a1e3"
    --bucketId 27327
```

- Download the file by its CID

```
npx @cere-ddc-sdk/cli download baebb4ide2ma4fvzr35kkgaq
lnh5lgeojwsyhp4n4k7jc2qgsdizcariizm ./picture.jpg
    --signer "hybrid label reunion only dawn maze asset
draft cousin height flock nation"
    --clusterId "0x825c4b2352850de9986d9d28568db6f0c023
a1e3"
    --bucketId 27327
```

Also, visit the examples directory for the most common runnable scenarios. You can also use the playground app to experiment with the SDK directly from your browser.

# 🔬 R&D

New storage architecture migration

# ❓ Questions

▼ Open questions:

▼ Closed questions:

# ▼ 📎 Additional links and references

# ▼ 📜 Archive

## Public Facing Desc:

[sync bloc]

The DDC SDK is a development kit used by developers to create applications that interact with the CERE infrastructure. It provides a set of modules and methods that allow seamless integration with the Cerebellum Network's decentralized data cloud (DDC).

▼ **Public Wiki don't move**

- Content-addressable storage
  Use when need simple upload data to DDC as one unit. Usage: JS
- File storage
  Use when need to upload large data by chunks and handle them like a file if need. Usage: JS
- Key-Value storage
  Use when need store data by key. Usage: JS

- Smart-contract
  Client for working with DDC Smart-Contract. Usage: JS
- DDC-client
  Simple client with all methods for working with DDC. Usage: JS

## Tests & Simulations

- [ ] DDC Client SDK Tests & Simulations